Добавь юнит тесты оформи проект на гит хаб

Да. Ниже — готовая структура проекта для GitHub с юнит-тестами. Проект называется **Responsibility Response Model (RRM)**.

Структура репозитория

```
responsibility-response-model/
├── rrm/
│   ├── __init__.py
│   ├── model.py
│   ├── scenarios.py
│   └── visualization.py
├── tests/
│   ├── __init__.py
│   ├── test_model.py
│   └── test_scenarios.py
├── .gitignore
├── requirements.txt
├── README.md
└── LICENSE
```

`pytest` автоматически находит файлы `test_*.py` и функции `test_*`; тесты принято хранить в отдельной папке `tests/`.[1] [2] [3]

Файл: rrm/model.py

```
# -*- coding: utf-8 -*-
"""Ядро универсальной модели ответственности."""

import numpy as np


def delay_coefficient(t, lam=0.15, critical_time=1.0):
    """
    Коэффициент задержки помощи.

    D(t) = 1, если t <= critical_time
    D(t) = exp(-lambda * (t - critical_time)), если t > critical_time
    """
    t = np.asarray(t, dtype=float)

    return np.where(
        t <= critical_time,
        1.0,
```

```
        np.exp(-lam * (t - critical_time))
    )


def support_function(t, base_support=60.0, decay=0.03):
    """
    Непрерывная поддержка во времени.

    S(t) = base_support * exp(-decay * t)
    """
    t = np.asarray(t, dtype=float)

    return base_support * np.exp(-decay * t)


def model_value(B, R, S, lam, critical_time=1.0, T=20.0, steps=1000):
    """
    V(T) = B - R + ∫ D(t) * S(t) dt
    """
    t = np.linspace(0, T, steps)
    dt = t[^1] - t[^0]

    D = delay_coefficient(t, lam, critical_time)
    support_integral = np.sum(D * S * dt)

    return B - R + support_integral
```

Файл: rrm/scenarios.py

```
# -*- coding: utf-8 -*-
"""Сценарии применения модели."""

from rrm.model import model_value, support_function


SCENARIOS = {
    "Человек-спасатель": {
        "B": 100.0,
        "R": 80.0,
        "S_base": 60.0,
        "S_decay": 0.03,
        "lambda": 0.20,
        "critical_time": 1.0
    },
    "Финансовый кризис": {
        "B": 100.0,
        "R": 90.0,
        "S_base": 70.0,
        "S_decay": 0.05,
        "lambda": 0.15,
        "critical_time": 2.0
    },
    "Нейросеть": {
        "B": 100.0,
        "R": 70.0,
```

```
        "S_base": 55.0,
        "S_decay": 0.08,
        "lambda": 0.30,
        "critical_time": 0.5
    }
}


def evaluate_scenarios(T=20.0):
    results = {}

    for name, params in SCENARIOS.items():
        t = np.linspace(0, T, 1000)

        S = support_function(
            t,
            base_support=params["S_base"],
            decay=params["S_decay"]
        )

        value = model_value(
            B=params["B"],
            R=params["R"],
            S=S,
            lam=params["lambda"],
            critical_time=params["critical_time"],
            T=T
        )

        results[name] = value

    return results
```

В начале `scenarios.py` добавь:

```
import numpy as np
```

Файл: rrm/visualization.py

```
# -*- coding: utf-8 -*-
"""Визуализация и анимация модели."""

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, FFMpegWriter, PillowWriter

from rrm.model import delay_coefficient, model_value, support_function
from rrm.scenarios import evaluate_scenarios


def plot_delay_curves():
    t = np.linspace(0, 20, 500)
```

```
    fast = delay_coefficient(t, lam=0.05)
    medium = delay_coefficient(t, lam=0.15)
    slow = delay_coefficient(t, lam=0.30)

    plt.figure(figsize=(10, 6))

    plt.plot(t, fast, label="Быстрая система: λ = 0.05", linewidth=2.5)
    plt.plot(t, medium, label="Средняя система: λ = 0.15", linewidth=2.5)
    plt.plot(t, slow, label="Медленная система: λ = 0.30", linewidth=2.5)

    plt.axvline(1, color="red", linestyle="--", label="Критическое время")

    plt.title("Коэффициент задержки помощи D(t)")
    plt.xlabel("Время задержки t")
    plt.ylabel("Эффективность помощи D(t)")
    plt.grid(True, alpha=0.3)
    plt.legend()

    plt.tight_layout()
    plt.savefig("1_delay_coefficient.png", dpi=160)
    plt.close()


def plot_scenario_comparison():
    results = evaluate_scenarios()

    names = list(results.keys())
    values = list(results.values())

    plt.figure(figsize=(10, 6))

    bars = plt.bar(names, values, color=["#2e86de", "#e67e22", "#8e44ad"])

    plt.axhline(0, color="black", linewidth=1)
    plt.title("Итоговая ценность системы V(T)")
    plt.ylabel("V(T)")
    plt.grid(axis="y", alpha=0.3)

    for bar, value in zip(bars, values):
        plt.text(
            bar.get_x() + bar.get_width() / 2,
            value + 1,
            f"{value:.1f}",
            ha="center",
            fontsize=11
        )

    plt.tight_layout()
    plt.savefig("2_system_comparison.png", dpi=160)
    plt.close()


def plot_reaction_speed():
    lambdas = np.linspace(0.01, 0.60, 200)
    values = []
```

```
    for lam in lambdas:
        t = np.linspace(0, 20, 1000)
        S = support_function(t, base_support=60.0, decay=0.03)

        value = model_value(
            B=100,
            R=80,
            S=S,
            lam=lam,
            critical_time=1.0,
            T=20.0
        )

        values.append(value)

    plt.figure(figsize=(10, 6))
    plt.plot(lambdas, values, color="#c0392b", linewidth=3)
    plt.axhline(0, color="black", linestyle="--")

    plt.title("Чем медленнее реакция, тем ниже V(T)")
    plt.xlabel("Скорость потери эффективности помощи λ")
    plt.ylabel("Итоговая ценность V(T)")
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig("3_reaction_speed.png", dpi=160)
    plt.close()


def plot_model_principle():
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.axis("off")

    text = (
        "RESPONSIBILITY RESPONSE MODEL\n\n"
        "V(T) = B(T) - R(T) + ∫ D(t) S(t) dt\n\n"
        "B(T) — ожидаемая польза\n"
        "R(T) — ожидаемый вред\n"
        "S(t) — поддержка и исправление\n"
        "D(t) — коэффициент задержки\n"
        "T — период ответственности\n\n"
        "Главный принцип:\n"
        "Чем позже помощь, тем меньше она искупает вред."
    )

    ax.text(
        0.5, 0.5, text,
        ha="center", va="center",
        fontsize=13,
        family="monospace"
    )

    plt.tight_layout()
    plt.savefig("4_model_principle.png", dpi=160)
    plt.close()
```

```
def create_animation(export_mp4=True, ffmpeg_available=True):
    T_MAX = 20.0
    FRAMES = 120
    LAMBDA = 0.18
    CRITICAL_TIME = 1.0

    t_values = np.linspace(0, T_MAX, 500)

    B = 100.0
    R = 80.0
    S_base = 60.0
    S_decay = 0.03

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))

    line_D, = ax1.plot([], [], color="#c0392b", linewidth=3)
    line_V, = ax2.plot([], [], color="#2e86de", linewidth=3)

    ax1.set_xlim(0, T_MAX)
    ax1.set_ylim(0, 1.1)
    ax1.set_xlabel("Время задержки t")
    ax1.set_ylabel("D(t)")
    ax1.set_title("Задержка помощи")

    ax2.set_xlim(0, T_MAX)
    ax2.set_ylim(-20, 100)
    ax2.set_xlabel("Время задержки t")
    ax2.set_ylabel("V(T)")
    ax2.set_title("Итоговая ценность системы")

    text = ax2.text(
        0.02, 0.90,
        "",
        transform=ax2.transAxes,
        fontsize=11,
        family="monospace"
    )

    def update(frame):
        current_t = T_MAX * frame / (FRAMES - 1)

        D_current = delay_coefficient(
            np.array([current_t]),
            lam=LAMBDA,
            critical_time=CRITICAL_TIME
        )[^0]

        S_current = support_function(
            np.array([current_t]),
            base_support=S_base,
            decay=S_decay
        )[^0]

        V_current = model_value(
            B=B,
```

```
            R=R,
            S=S_current,
            lam=LAMBDA,
            critical_time=CRITICAL_TIME,
            T=current_t
        )

        t_plot = t_values[t_values <= current_t]

        D_plot = delay_coefficient(
            t_plot,
            lam=LAMBDA,
            critical_time=CRITICAL_TIME
        )

        V_plot = []
        for time_point in t_plot:
            S_point = support_function(
                np.array([time_point]),
                base_support=S_base,
                decay=S_decay
            )[^0]

            value = model_value(
                B=B,
                R=R,
                S=S_point,
                lam=LAMBDA,
                critical_time=CRITICAL_TIME,
                T=time_point
            )

            V_plot.append(value)

        line_D.set_data(t_plot, D_plot)
        line_V.set_data(t_plot, V_plot)

        text.set_text(
            f"t = {current_t:.2f}\n"
            f"D(t) = {D_current:.3f}\n"
            f"S(t) = {S_current:.2f}\n"
            f"V(T) = {V_current:.2f}"
        )

        return line_D, line_V, text

    ani = FuncAnimation(
        fig,
        update,
        frames=FRAMES,
        interval=60,
        blit=False,
        repeat=True
    )

    if export_mp4 and ffmpeg_available:
```

```
        writer = FFMpegWriter(fps=30, bitrate=2400)
        ani.save("model_delay_animation.mp4", writer=writer, dpi=150)
        return "model_delay_animation.mp4"

    writer = PillowWriter(fps=15)
    ani.save("model_delay_animation.gif", writer=writer, dpi=120)
    return "model_delay_animation.gif"
```

Файл: tests/test_model.py

```
# -*- coding: utf-8 -*-

import numpy as np

from rrm.model import (
    delay_coefficient,
    model_value,
    support_function
)


def test_delay_coefficient_is_one_before_critical_time():
    result = delay_coefficient(0.5, lam=0.2, critical_time=1.0)
    assert result == 1.0


def test_delay_coefficient_decreases_after_critical_time():
    early = delay_coefficient(2.0, lam=0.2, critical_time=1.0)
    late = delay_coefficient(5.0, lam=0.2, critical_time=1.0)

    assert early > late
    assert late < 1.0


def test_delay_coefficient_zero_lambda_is_constant():
    result = delay_coefficient(10.0, lam=0.0, critical_time=1.0)
    assert result == 1.0


def test_support_function_decreases_over_time():
    early = support_function(0.0)
    late = support_function(10.0)

    assert early > late
    assert late >= 0.0


def test_model_value_positive_without_risk():
    value = model_value(
        B=100,
        R=0,
        S=50,
        lam=0.1,
        critical_time=1.0,
        T=10.0
```

```
    )

    assert value > 0


def test_model_value_decreases_with_delay():
    fast = model_value(
        B=100,
        R=80,
        S=60,
        lam=0.05,
        critical_time=1.0,
        T=20.0
    )

    slow = model_value(
        B=100,
        R=80,
        S=60,
        lam=0.30,
        critical_time=1.0,
        T=20.0
    )

    assert fast > slow


def test_model_value_without_support_equals_b_minus_r():
    value = model_value(
        B=100,
        R=80,
        S=0,
        lam=0.2,
        critical_time=1.0,
        T=10.0
    )

    assert value == 20.0
```

Файл: tests/test_scenarios.py

```
# -*- coding: utf-8 -*-

from rrm.scenarios import evaluate_scenarios


def test_all_scenarios_return_values():
    results = evaluate_scenarios()

    assert len(results) == 3

    for value in results.values():
        assert isinstance(value, float)
```

```
def test_financial_crisis_has_lower_value_than_human_case():
    results = evaluate_scenarios()

    assert results["Финансовый кризис"] < results["Человек-спасатель"]


def test_neural_network_has_positive_value():
    results = evaluate_scenarios()

    assert results["Нейросеть"] > 0


def test_scenario_values_are_finite():
    results = evaluate_scenarios()

    for value in results.values():
        assert np.isfinite(value)
```

В начале `test_scenarios.py` добавь:

```
import numpy as np
```

Файл: requirements.txt

```
numpy
matplotlib
pillow
pytest
```

Файл: .gitignore

```
__pycache__/
*.py[cod]
*.egg-info/
.venv/
venv/
.env

# Результаты визуализации
*.png
*.gif
*.mp4

# Системные файлы
.DS_Store
```

Файл: README.md

```
# Responsibility Response Model (RRM)

Универсальная модель ответственности:

V(T) = B(T) - R(T) + ∫ D(t) * S(t) dt

Где:
- B(T) — ожидаемая польза
- R(T) — ожидаемый вред
- S(t) — поддержка, компенсация, исправление
- D(t) — коэффициент задержки помощи
- T — период ответственности

D(t) = exp(-lambda * t)

## Что моделирует

- Человека, который спасает другого и получает травму.
- Государство, которое реагирует на кризис.
- Финансовую систему во время кризиса.
- Нейросеть, которая ошибается и должна исправлять вред.
- Любую систему, где задержка помощи увеличивает ущерб.

## Установка

```bash
python -m venv .venv
source .venv/bin/activate  # Linux/macOS
.venv\Scripts\activate     # Windows

pip install -r requirements.txt
```

Запуск

```
python -m rrm.main
```

Тесты

```
pytest
```

Главная идея

Чем позже помощь, тем меньше она искупает вред.

Ответственность — это не разовый поступок,
а непрерывная реакция во времени.
```

```
## Файл: LICENSE

Добавь файл `LICENSE` с текстом лицензии MIT, если хочешь открыть проект.

## Как создать GitHub-репозиторий

1. Создай пустой репозиторий на GitHub с именем:

```text
responsibility-response-model
```

2. Не добавляй на GitHub README, `.gitignore` или LICENSE при создании — их лучше добавить локально и запушить вместе с проектом. [4] [5] [6]

3. В терминале, внутри папки проекта, выполни:

```
git init
git add .
git commit -m "Initial commit: Responsibility Response Model"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/responsibility-response-model
git push -u origin main
```

GitHub рекомендует инициализировать локальный репозиторий через `git init`, добавить файлы, сделать commit и привязать удалённый репозиторий через `git remote add origin`. [6] [4]

Как запустить тесты

Внутри папки проекта:

```
pip install -r requirements.txt
pytest
```

`pytest` найдёт тесты автоматически благодаря именам `test_*.py` и функциям `test_*`. [7] [8] [1]

Главная формула проекта

$$\boxed{V(T) = B(T) - R(T) + \int_0^T e^{-\lambda t} S(t) dt}$$

Финальный принцип

Любая система ответственна не только за решение, но и за скорость и качеств

Теперь у тебя не просто скрипт, а **готовый GitHub-проект с модульной структурой, тестами, документацией и визуализацией.** [5] [1] [4]
```

⁂

1. https://docs.pytest.org/en/stable/getting-started.html
2. https://learn.scientific-python.org/development/guides/pytest/
3. https://learn.scientific-python.org/development/principles/testing/
4. https://docs.github.com/en/get-started/using-git/about-git
5. https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository
6. https://docs.github.com/en/enterprise/2.13/user/articles/adding-an-existing-project-to-github-using-the-command-line
7. https://docs.pytest.org/en/8.0.x/
8. https://docs.pytest.org/en/stable/explanation/goodpractices.html
9. https://docs.pytest.org/en/stable/
10. https://docs.pytest.org/en/stable/contents.html
11. https://docs.github.com/en/desktop/overview/creating-your-first-repository-using-github-desktop
12. https://docs.github.com/en/repositories/creating-and-managing-repositories/quickstart-for-repositories
13. https://docs.pytest.org/_/downloads/en/stable/pdf/
14. https://docs.github.com/api/article/body?pathname=/en/repositories/creating-and-managing-repositories/creating-a-new-repository
15. https://docs.github.com/ru/enterprise-server@3.10/get-started/using-git/about-git
16. https://www.resus.org.uk/professional-library/2025-resuscitation-guidelines/ethics-guidelines